

17173系统部

17173系统部团队技术博客

LXC基础学习教程

陈承 (cyent@163.com ,QQ:57237382)

Create date: 2013/07/12 14:40 Last Update: 2014/10/13 09:37

1. LXC介绍

2. liblxc介绍与使用

- 2.1. 介绍
- 2.2. 帮助
 - 2.2.1. man
 - 2.2.2. -help
 - 2.2.3. 推荐网址
- 2.3. 软件安装
 - 2.3.1. 操作系统及lxc版本
 - 2.3.1.1. 安装依赖包
 - 2.3.1.2. 方法1: RPM安装
 - 2.3.1.3. 方法2: 编译安装
 - 2.3.1.4. 方法3: (规范安装) yum安装
- 2.4. 宿主配置
 - 2.4.1. 宿主服务器 – libcgroup
 - 2.4.2. 宿主服务器 – 开启网桥模式
 - 2.4.3. 宿主服务器 – cgroup设置
 - 2.4.4. 宿主服务器 – 挂载cgroup实现资源控制
- 2.5. 构建容器
 - 2.5.1. 3要素
 - 2.5.1.1. config
 - 2.5.1.2. fstab
 - 2.5.1.3. rootfs
 - 2.5.2. 从零构建容器
 - 2.5.2.1. rpm制作模板
 - 2.5.2.2. 解压rpm
 - 2.5.2.3. 修改rootfs文件系统模板的yum源
 - 2.5.2.4. 安装rootfs必要的软件及基础模块
 - 2.5.2.5. 初始化rootfs

- 2.5.2.5.1. lxc rootfs基础配置
 - 2.5.2.5.2. 为lxc创建必要的设备节点
 - 2.5.2.5.3. 创建一些基本的目录
 - 2.5.2.5.4. 用户环境配置文件
 - 2.5.2.5.5. 创建网络文件
 - 2.5.2.5.6. 增加容器启动初始化脚本
- 2.6. 启动容器
 - 2.6.1. 关闭lxc实例
 - 2.6.2. 宿主服务器查看lxc实例进程
 - 2.6.3. 宿主服务器动态限制lxc资源
 - 2.6.4. lxc实例控制台
 - 2.6.5. 内核是否支持lxc运行
 - 2.6.6. 实例里的网络连接状态
 - 2.6.7. 实例里的进程状态
 - 2.6.8. 实例状态监控
 - 2.6.9. LXC操作指令详解
- 2.7. 网络类型
 - 2.7.1. 物理（phys）网络
 - 2.7.2. 配置桥接（veth）网络
 - 2.7.3. 配置macvlan网络
 - 2.7.3.1. Macvlan分为三种类型:
 - 2.7.3.2. 宿主机MAC Bridge配置
 - 2.7.3.3. 配置Vlan网络

3. LXC脚本

- 3.1. 介绍
- 3.2. 功能说明
- 3.3. 安装部署
- 3.4. 容器配置规范
 - 3.4.1. config
 - 3.4.2. rootfs
- 3.5. 脚本使用
 - 3.5.1. 宿主机配置
 - 3.5.1.1. 内核支持检查
 - 3.5.1.2. LXC版本
 - 3.5.1.3. CGROUP配置与网桥模式自动配置
 - 3.5.2. 容器配置
 - 3.5.2.1. 容器创建
 - 3.5.2.2. 容器删除
 - 3.5.2.3. 容器初始化
 - 3.5.3. 容器服务控制
 - 3.5.3.1. 启动
 - 3.5.3.2. 停止
 - 3.5.3.3. 状态
 - 3.5.3.4. 销毁
 - 3.5.3.5. 所有容器状态

- 3.5.3.6. 控制台
- 3.5.3.7. 冻结
- 3.5.3.8. 解冻
- 3.5.3.9. 资源查看与限制
- 3.5.4. 实用功能
 - 3.5.4.1. 进程查看
 - 3.5.4.2. 网络连接查看
 - 3.5.4.3. 容器内存使用情况查看

4. 神器

- 4.1. nsenter（无需sshd、无需attach也可以登录容器）

5. FAQ

- 5.1. 容器启动报错
 - 5.1.1. Too many open files – failed to inotify_init
- 5.2. resin/httpd等服务非正常关闭, 导致容器出错
- 5.3. 容器名被占用的解决办法
 - 5.3.1. 更换容器名（不建议）
 - 5.3.2. 强制删除
- 5.4. LXC宿主上做LVS时，真实服务器不能是同一台宿主机下面的LXC容器
- 5.5. 网卡名长度限制
- 5.6. 执行dmidecode报错"/dev/mem: No such file or directory"
- 5.7. 加载fuse报错
- 5.8. 宿主机问题导致的容器使用异常
 - 5.8.1. TTY问题
- 5.9. 添加tty, 允许更多的终端登录
- 5.10. 服务检查
- 5.11. UTF中文字符处理
- 5.12. 使用RHEL发行版的yum资源
- 5.13. 操作系统时区与硬件时钟
- 5.14. 预读取/预处理脚本删除
- 5.15. 使用ssh/scp等出现问题
- 5.16. 桥接网卡MAC绑定
- 5.17. lxc-monitor工作异常
- 5.18. OOM问题
- 5.19. 修改容器里内核参数是否会影响宿主
- 5.20. Capabilities（Linux能力）
- 5.21. 容器已启动，但console终端控制台空白无显示
- 5.23. 容器里使用nfs挂载
- 5.24. 容器充当nfs server
- 5.25. 容器重启导致宿主终端报错
- 5.26. lxc-start时候宿主内核crash
- 5.27. 容器内无法使用iptables(模块加载失败)
- 5.28. udev导致容器异常

tag: cloud, virtual, lxc

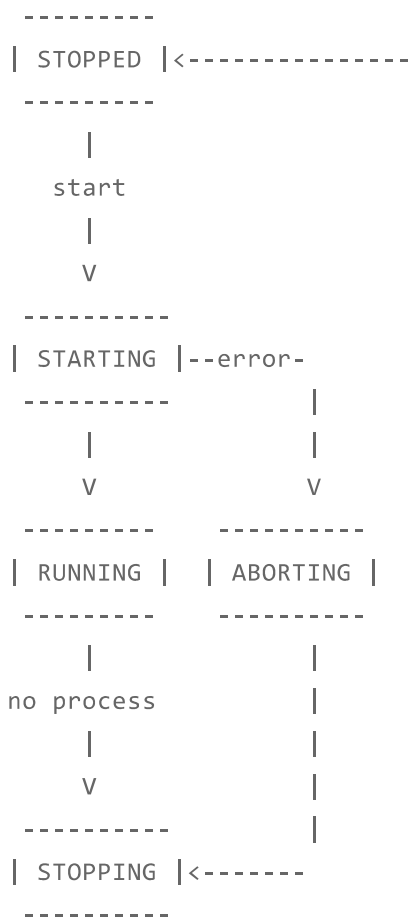
1. LXC介绍

- LXC是Linux containers的简称，是一种基于容器的操作系统层级的虚拟化技术。
- LXC可以在操作系统层次上为进程提供虚拟的执行环境，一个虚拟的执行环境就是一个容器，同时可以为容器绑定特定的cpu和memory节点，分配特定比例的cpu时间、IO时间，限制可以使用的内存大小（包括内存和是swap空间），提供device访问控制，提供独立的namespace（网络、pid、ipc、mnt、uts）。
- LXC主要包含三个方面：cgroup、network和rootfs。
 - Cgroups是control groups的缩写，是Linux内核提供了一种可以限制、记录、隔离进程组（process groups）所使用的物理资源（如：cpu,memory,IO等等）的机制。LXC主要通过cgroup实现对于资源的管理和控制。
 - Network是为容器提供的网络环境设置。
 - rootfs用于指定容器的虚拟根目录，设定此项以后，容器内所有进程将会把此目录根目录，不能访问此目录之外的路径，相当于chroot的效果。
- 生命周期和状态

CONTAINER LIFE CYCLE

When the container is created, it contains the configuration information. When a process is launched, the container will be starting and running. When the last process running inside the container exits, the container is stopped.

In case of failure when the container is initialized, it will pass through the aborting state.



另外：从lxc-wait帮助里可以看到有7种状态：STOPPED，STARTING，RUNNING，STOPPING，ABORTING，FREEZING，FROZEN。

2. liblxc介绍与使用

2.1. 介绍

- 已知的实现lxc的解决方案有2个：liblxc与libvirt，本文仅介绍liblxc的使用

2.2. 帮助

2.2.1. man

- LXC概要：man lxc
- LXC配置文件：man lxc.conf
- LXC操作指令：
 - man lxc-cgroup
 - man lxc-checkpoint
 - man lxc-console
 - man lxc-create
 - man lxc-destroy
 - man lxc-execute
 - man lxc-freeze
 - man lxc-kill
 - man lxc-ls
 - man lxc-monitor
 - man lxc-ps
 - man lxc-restart
 - man lxc-start
 - man lxc-stop
 - man lxc-unfreeze
 - man lxc-wait
- Capabilities（Linux能力）：
 - man capabilities

2.2.2. -help

- 操作指令均支持加上-help选项显示简要帮助信息

2.2.3. 推荐网址

- LXC介绍
 - (英文) (liblxc官网) <http://linuxcontainers.org/>
 - (英文) <http://lxc.teegra.net/>
 - (中文) <http://baike.baidu.com/link?url=huNRgE-YXhPx8jOQ6YIO3kgh80iChcp77-08j37i73bBq5tNS6B2TDnJP-NkjZnhzgTnqE0TaUVoe-EiPyq-jK>
 - (中文) (Linux Container概述, 以及其存在的问题和挑战) (视频,PPT) <http://www.infoq.com/cn/presentations/linux-container-overview>
 - (英文) https://wiki.archlinux.org/index.php/Linux_Containers
 - (英文) (Lxc/Installation/Guest/Centos/6) <http://wiki.1tux.org/wiki/Lxc/Installation/Guest/Centos/6>
 - (中文) (基于CHROOT发展而来) <http://blogs.ejb.cc/archives/5462/lxc-linux-container-early-glance>
 - (英文) <http://wiki.gentoo.org/wiki/LXC>
 - (中文) <http://www.ibm.com/developerworks/cn/linux/l-lxc-containers/>
 - (英文) <http://uncommonsense-uk.com/2012/virtual-machine-stacking-using-lxc-on-top-of-esx/>
 - (中文) <http://wenku.baidu.com/view/eabc5647a8956bec0975e34e.html>
- LXC原理框架
 - (中文) <http://www.tuicool.com/articles/ArimMz>
- 容器与应用生命周期的耦合
 - (中文) <http://www.cnblogs.com/lisperl/archive/2012/06/20/2556523.html>
- 桥接网卡MAC动态变更
 - (英文) <http://backreference.org/2010/07/28/linux-bridge-mac-addresses-and-dynamic-ports/>
- CentOS6之下的lxc-0.7.5安装使用教程
 - (英文) <http://whistl.com/index.php/blog/2011/08/28/linux-containers-under-centos-6>
- LXC网络带宽限制
 - (中文) http://www.icsmile.com/2013/05/14/lxc_network.html
- 容器多网卡
 - (英文) <http://www.boxtricks.com/multiple-network-interfaces-lxc-container/>
- devpts
 - <https://www.kernel.org/doc/Documentation/filesystems/devpts.txt>
- API
 - (英文) <http://www.stgraber.org/2012/09/28/introducing-the-python-lxc-api/>
 - (英文) <https://pypi.python.org/pypi?:action=search&term=LXC>
 - (英文) <https://github.com/cloud9ers/pylxc>
- 操作指令
 - (英文) <http://blog.chinaunix.net/uid-434226-id-3425846.html>
 - (中文) http://os.chinaunix.net/a2009/0302/1049/000001049064_2.shtml
 - (中文) <http://www.cnblogs.com/lisperl/archive/2012/04/13/2446179.html>
- Linux能力 (capability)
 - (中文) <http://www.cnblogs.com/iamfy/archive/2012/09/20/2694977.html>
- LXC高级功能第三方实现
 - (中文) (docker,dockerlite,routeflow) <http://blog.csdn.net/quqi99/article/details/9532105>
 - (英文) (Docker) <http://blog.dotcloud.com/bootstrapping-the-open-source-sandbox-on-top-of-docker>
 - (英文) (Docker) <http://www.docker.io/gettingstarted/>
- libvirt

- (英文) (libvirt之lxc官网) <http://libvirt.org/drmxc.html>
- (中文) (使用libvirt管理lxc之virsh篇) http://blog.sina.com.cn/s/blog_999d1f4c0101dxad.html
- (中文) (用libvirt管理lxc) <http://hi.baidu.com/anatacollin/item/a7ae54ca7110b92fef466515>
- (中文) (基于libvirt的kvm虚拟机热迁移) <http://www.libaoyin.com/2013/05/19/libvirt-live-migration-without-shared-storage/>
- (中文) (libvirt-迁移) <http://romanker.blog.163.com/blog/static/18564353020118845456750/>
- (中文) (配置支持LXC的CentOS 6) <http://wiki.centos.org/zh/HowTos/LXC-on-CentOS6>
- (英文) <https://www.berrange.com/posts/2009/12/03/using-cgroups-with-libvirt-and-lxc-kvm-guests-in-fedora-12/>
- CGROUP
 - (中文) (LXC CPU资源隔离与动态调整测试) <http://speakingbaicai.blog.51cto.com/5667326/1162693>
 - (中文) (I/O控制 blkio) <http://blog.tao.ma/?p=43>
 - (中文) (I/O控制 blkio)
http://www.07net01.com/linux/Cgroup_blkio_I_O_kongzhi_456892_1372160754.html
 - (中文) (I/O控制 blkio) <http://www.elmerzhang.com/2012/12/cgroups-learning-6-blkio-subsystem/>
 - (中文) (使用cgroup限制java使用的内存量-思路) <http://blog.csdn.net/sraing/article/details/8925977>
- Linux系统启动过程
 - (中文) (详解linux系统的启动过程及系统初始化) <http://www.aikaiyuan.com/5135.html>

2.3. 软件安装

2.3.1. 操作系统及lxc版本

- OS: RHEL 6.4
- LXC: lxc-0.7.5
- lxc-0.7.5以上版本在RHEL 6.3-6.4运行碰到问题

2.3.1.1. 安装依赖包

- 这里直接使用yum进行安装

```
# yum install libcap-devel
# yum install docbook2X
```

2.3.1.2. 方法1: RPM安装

- 生成rpm

```
# tar -zxvf lxc-0.7.5.tar.gz
# cd lxc-0.7.5
# ./configure
```

```
# make rpm
.....
Wrote: /root/rpmbuild/SRPMS/lxc-0.7.5-1.src.rpm
Wrote: /root/rpmbuild/RPMS/x86_64/lxc-0.7.5-1.x86_64.rpm
Wrote: /root/rpmbuild/RPMS/x86_64/lxc-devel-0.7.5-1.x86_64.rpm
Wrote: /root/rpmbuild/RPMS/x86_64/lxc-debuginfo-0.7.5-1.x86_64.rpm
.....
```

- 安装rpm

```
# cd /root/rpmbuild/RPMS/x86_64/
# rpm -ivh *
```

2.3.1.3. 方法2：编译安装

- 编译安装

```
# tar -zxvf lxc-0.7.5.tar.gz
# cd lxc-0.7.5
# ./configure
make && make install
```

- 关于`--prefix`注意点

1. 若使用`./configure`不带`--prefix`，则`lxc-start`、`lxc-console`连接的socket为`@/var/lib/lxc/v-fz-qa-30/command`
2. 若使用`./configure --prefix=/opt/17173/lxc-0.7.5`，则`lxc-start`、`lxc-console`连接的socket为`@/opt/17173/lxc-0.7.5/var/lib/lxc/v-fz-qa-43/command`

默认情况下不通过`lxc-create`命令创建容器，而是手动创建，因此必须保证`/var/lib/lxc/`目录存在且为空；指定了`--prefix`也一样，必须保证`/opt/17173/lxc-0.7.5/var/lib/lxc/`目录存在且为空。因为`lxc-ls`里以`$prefix/var/lib/lxc`存在为执行前提，然后会`ls $prefix/var/lib/lxc/`

2.4. 宿主配置

2.4.1. 宿主服务器 – libcgroup

- cgroup使用liblxc实现的cgroup，而不用libcgroup，但是libcgroup集成的删除cgroup命名空间的操作指令（cgdelete）是liblxc不具备的（作用，强制删除无用的/cgroup/name命名空间），因此要装，但不使用除cgdelete以外的功能。

```
# yum install libcgroup
```

默认/etc/init.d/cgconfig和/etc/init.d/cgred都是stop的状态，并且chkconfig也是off，不过为了保险起见，建议将这2个服务手动再次关闭并确保chkconfig为off状态

也可以使用rmmdir来删除/cgroup下的容器（前提是cgroup.procs里的进程都已杀掉）

libvirt依赖libcgroup，而libcgroup不依赖libvirt，安装libcgroup。建议yum remove libvirt，然后再yum install libcgroup

注意：cfconfig stop将会执行cgclear（卸载cgroup，但仅从/proc/mounts里去除，并未从/etc/mtab，即mount命令输出中去除），执行这条命令需要小心，如果状态已经是stopped的，就没必要再次执行

注意：卸载libcgroup包将会删除/cgroup目录，因此没有特殊需求时不要这么做

2.4.2. 宿主服务器 – 开启网桥模式

- 添加桥接网卡

```
# vim ifcfg-br0
DEVICE="br0"
ONBOOT="yes"
TYPE="Bridge"
BOOTPROTO="static"
IPADDR="192.168.1.1"
NETMASK="255.255.255.0"
GATEWAY="192.168.1.254"
```

- 修改em2网卡

```
# vim ifcfg-em2
DEVICE="em2"
BRIDGE="br0"
ONBOOT="yes"
BOOTPROTO="none"
NM_CONTROLLED="no"
TYPE="Ethernet"
```

- 重启网络

```
# /etc/rc.d/init.d/network restart
Shutting down interface em2:  bridge br0 does not exist!
[ OK ]
Shutting down loopback interface:
[ OK ]
Bringing up loopback interface:
[ OK ]
Bringing up interface em2:
[ OK ]
Bringing up interface br0:
[ OK ]
```

2.4.3. 宿主服务器 – cgroup设置

- 关闭libcgroup集成的cfconfig和cgred服务

```
# checkconfig --level 35 cgconfig off
# checkconfig --level 35 cgred off
# /etc/rc.d/init.d/cfconfig stop # 注意: cfconfig stop将会执行cgclear（卸载/cgroup
， 但仅从/proc/mounts里去除，并未从/etc/mtab，即mount命令输出中去除），执行这条命令需要小
心，如果状态已经是stopped的，就没必要再次执行
# /etc/rc.d/init.d/cgred stop
```

2.4.4. 宿主服务器 – 挂载cgroup实现资源控制

- 创建/cgroup, 编辑/etc/fstab

```
# mkdir /cgroup
# vim /etc/fstab
```

```
none /cgroup cgroup defaults 0 0
```

2.5. 构建容器

2.5.1. 3要素

- 一个完整的LXC容器必须包含以下2个文件和1个目录
 - config
 - fstab
 - rootfs

2.5.1.1. config

- 格式: key = value

```
lxc.network.type = veth
```

- config内容与分类
 - ARCHITECTURE
 - lxc.arch # 定义架构类型, 通常是x86_64
 - HOSTNAME
 - lxc.utsname # 定义本lxc实例的名称
 - NETWORK
 - lxc.network.type # 定义网络类型
 - lxc.network.flags # 此项不生效, 因为当前只支持up, 默认也是up
 - lxc.network.link # 定义宿主服务器的网络设备进行桥接 (若lxc.network.type设置为veth)
 - lxc.network.name # 定义lxc实例的网卡名称 (不超过15位)
 - lxc.network.hwaddr # 定义lxc实例的网卡硬件地址
 - lxc.network.veth.pair # 定义宿主服务器可见的网络设备名称, ifconfig可见
 - lxc.network.ipv4 # 定义lxc实例的IPv4地址
 - lxc.network.ipv6 # 定义lxc实例的IPv6地址
 - lxc.network.script.up
 - NEW PSEUDO TTY INSTANCE (DEVPTS)
 - lxc.pts # 伪终端数量
 - CONTAINER SYSTEM CONSOLE
 - lxc.console # console记录输出至文件里, 也可通过lxc-start的-c选项来指定文件
 - CONSOLE THROUGH THE TTYS
 - lxc.tty # tty数量
 - MOUNT POINTS
 - lxc.mount # fstab文件路径
 - lxc.mount.entry # fstab文件内容直接写于此, 替代lxc.mount指定文件路径方式

- ROOT FILE SYSTEM
 - lxc.rootfs # 指定rootfs路径
 - lxc.rootfs.mount
 - lxc.pivotdir
- CONTROL GROUP
 - lxc.cgroup.subsystem # CGROUP配置
- CAPABILITIES
 - lxc.cap.drop # 移除指定的容器的Linux能力

2.5.1.2. fstab

- 格式: 和系统的/etc/fstab相同
- 示例

```
proc    /opt/lxc/template/rootfs/proc proc    nodev,noexec,nosuid 0 0
sysfs   /opt/lxc/template/rootfs/sys sysfs   defaults             0 0
tmpfs   /opt/lxc/template/rootfs/dev/shm tmpfs   defaults             0 0
```

2.5.1.3. rootfs

- 容器文件直接存储在宿主文件系统上
- 示例

```
[ 11:13:00-root@localhost:rootfs ]#ls
bin boot dev etc home lib lib64 media mnt opt proc root sbin selinu
x  srv  sys  tmp  usr  var
```

2.5.2. 从零构建容器

2.5.2.1. rpm制作模板

- 这里采用简单的rpm安装方法, 和kickstart、cdrom安装操作系统的流程几乎一致, 缺少了内核和驱动加载的初始化过程而已

```
# mkdir -p /opt/rhel6.4fs
# cd /opt/rhel6.4fs
# wget http://xxxxxxxx/pub/os/RedHat/enterprise/x86_64/6ASU4/disk/Packages/redh
at-release-server-6Server-6.4.0.4.el6.x86_64.rpm
```


2.5.2.2. 解压rpm

- 解压下载的rpm

```
# rpm2cpio redhat-release-server-6Server-6.3.0.3.el6.x86_64.rpm | cpio -idm
```

2.5.2.3. 修改rootfs文件系统模板的yum源

- 这里直接使用了17173内部的centos的yum源, 装起来怪怪的, 不过可以用

```
# cp /etc/yum.repos.d/17173-rhel6.repo /opt/lxc/rhel6.4fs/etc/yum.repos.d/  
# cd /opt/lxc/rhel6.4fs/etc/yum.repos.d/  
# vim 17173-rhel6.repo  
%s/\$releasever/6/g
```

2.5.2.4. 安装rootfs必要的软件及基础模块

- 安装到定义的rootfs模板路径

```
# yum --installroot=/opt/rhel6.4fs install lrzsz dhclient openssh-server passwd  
rsyslog vim-minimal vixie-cron wget which yum  
# yum --installroot=/opt/rhel6.4fs groupinstall base
```

2.5.2.5. 初始化rootfs

2.5.2.5.1. lxc rootfs基础配置

- chroot到rootfs里

```
# chroot /opt/rhel6.4fs
```

- 设置rootfs root密码

```
# passwd
```

2.5.2.5.2. 为lxc创建必要的设备节点

- 这里指定定义了一个tty, 试图增加过报错了, 这次先忽略

```
# rm -f /dev/null
# mknod -m 666 /dev/null c 1 3
# mknod -m 666 /dev/zero c 1 5
# mknod -m 666 /dev/urandom c 1 9
# ln -s /dev/urandom /dev/random
# mknod -m 600 /dev/console c 5 1
# mknod -m 660 /dev/tty c 5 0
# mknod -m 660 /dev/tty1 c 4 1
# mknod -m 660 /dev/tty2 c 4 2
# mknod -m 660 /dev/tty3 c 4 3
# mknod -m 660 /dev/tty4 c 4 4
# mknod -m 640 /dev/mem c 1 1
# chown root:tty /dev/tty
# chown root:tty /dev/tty1
# chown root:tty /dev/tty2
# chown root:tty /dev/tty3
# chown root:tty /dev/tty4
# chown root:kmem /dev/mem
```

2.5.2.5.3. 创建一些基本的目录

- 系统挂载需要使用的基础目录

```
# mkdir -p /dev/shm
# chmod 1777 /dev/shm
# mkdir -p /dev/pts
# chmod 755 /dev/pts
```

2.5.2.5.4. 用户环境配置文件

- root用户的基础登录环境

```
# cp -a /etc/skel/. /root/
```

2.5.2.5.5. 创建网络文件

- 必要的dns、网络、fstab等

```
# cat > /etc/resolv.conf << END
# fuzhou tele DNS
nameserver 218.85.157.99
nameserver 218.85.152.99
END

# cat /etc/hosts << END
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.0.1 lxc.local.17173.com
END

# cat /etc/sysconfig/network << END
NETWORKING=yes
HOSTNAME=lxc.local.17173.com
END

# cat > /etc/fstab << end
/dev/root / rootfs defaults 0 0
none /dev/shm tmpfs nosuid,nodev 0 0
end

# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE=eth0
BOOTPROTO=none
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.4.1
NETMASK=255.255.255.0

# cat /etc/sysconfig/network-scripts/route-eth0
192.168.1.0/24 via 192.168.1.254
192.168.2.0/24 via 192.168.1.254
```

2.5.2.5.6. 增加容器启动初始化脚本

- 由于容器的特殊性，需要添加以下配置文件至容器中

```
cat > /etc/init/lxc-sysinit.conf << END
start on startup
```

```
env container

pre-start script
    if [ "x${container}" != "xlxc" -a "x${container}" != "xlibvirt" ]; then
        stop;
    fi
    initctl start tty TTY=console
    rm -f /var/lock/subsys/*
    rm -f /var/run/*.pid
    rm -f /etc/mtab
    telinit 3
    if [ ! -e /etc/mtab ]; then
        echo '/dev/root / rootfs defaults 0 0' > /etc/mtab
        echo 'none /dev/shm tmpfs rw,nosuid,nodev 0 0' >> /etc/mtab
    else
        grep -q 'rootfs' /etc/mtab
        [ $? -eq 1 ] && echo '/dev/root / rootfs defaults 0 0' >> /etc/mtab
        grep -q 'tmpfs' /etc/mtab
        [ $? -eq 1 ] && echo 'none /dev/shm tmpfs rw,nosuid,nodev 0 0' >> /e
tc/mtab
        grep -q 'devpts' /etc/mtab
        [ $? -eq 0 ] && sed -i '/devpts/d' /etc/mtab
    fi
    rm -f /sbin/reboot /sbin/poweroff /sbin/halt /sbin/shutdown
    rm -f /etc/rc.d/rc0.d/* /etc/rc.d/rc6.d/*
    rm -f /usr/bin/halt /usr/bin/poweroff /usr/bin/reboot
    exit 0;
end script
EOF
```

2.6. 启动容器

- 启动调试功能的lxc容器, 主要进行调试, 看看有什么报错

```
lxc-start -n lxc-test1 -f /opt/lxc/test1/config -o /opt/lxc/test1/test1.log -l d
ebug
```

- 用daemon方式启动

```
lxc-start -n lxc-test1 -f /opt/lxc/test1/config -o /opt/lxc/test1/test1.log -d
```

2.6.1. 关闭lxc实例

- 仅仅是poweroff这个实例

```
lxc-stop -n lxc-test1
```

2.6.2. 宿主服务器查看lxc实例进程

- lxc在宿主服务器的状态

```
# lxc-info -n test1
state:    RUNNING
pid:      11695
```

2.6.3. 宿主服务器动态限制lxc资源

- lxc-cgroup限制, 查看现在的lxc实例使用的CPU

```
# lxc-cgroup -n test1 cpuset.cpus
2-5
```

- 动态修改实例CPU使用

```
# lxc-cgroup -n test1 cpuset.cpus "0-3"
# lxc-cgroup -n test1 cpuset.cpus
0-3
```

- 也可以在cgroup直接修改

```
# echo "0-1" > /cgroup/test1/cpuset.cpus
# lxc-cgroup -n test1 cpuset.cpus
0-1
```

2.6.4. lxc实例控制台

- 进入tty控制lxc实例

```
# lxc-console -n test1
Red Hat Enterprise Linux Server release 6.4 (Santiago)
Kernel 2.6.32-358.el6.x86_64 on an x86_64

test1 login:
```

- 退出tty控制台, 和退出screen相似

```
Ctrl a + q
```

- 远程tty

```
ssh -t remote_host lxc-console -n test
```

若不加-t参数会报错:

```
lxc-console: '0' is not a tty
lxc-console: failed to setup tios
```

2.6.5. 内核是否支持lxc运行

- lxc-checkconfig

```
# lxc-checkconfig
Kernel config /proc/config.gz not found, looking in other places...
Found kernel config file /boot/config-2.6.32-358.18.1.el6.x86_64
--- Namespaces ---
Namespaces: enabled
Utsname namespace: enabled
Ipc namespace: enabled
Pid namespace: enabled
User namespace: enabled
Network namespace: enabled
Multiple /dev/pts instances: enabled
```

```
--- Control groups ---
Cgroup: enabled
Cgroup namespace: enabled
Cgroup device: enabled
Cgroup sched: enabled
Cgroup cpu account: enabled
Cgroup memory controller: enabled
Cgroup cpuset: enabled

--- Misc ---
Veth pair device: enabled
Macvlan: enabled
Vlan: enabled
File capabilities: enabled
enabled

Note : Before booting a new kernel, you can check its configuration
usage : CONFIG=/path/to/config /usr/bin/lxc-checkconfig
```

2.6.6. 实例里的网络连接状态

- lxc-netstat

```
# lxc-netstat -n foo -lnpu
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         Stat
e      PID/Program name
tcp    0      0 0.0.0.0:80              0.0.0.0:*               LIST
EN     2831/nginx
tcp    0      0 0.0.0.0:22              0.0.0.0:*               LIST
EN     2797/sshd
```

2.6.7. 实例里的进程状态

- lxc-ps

```
# lxc-ps -n foo -- aux
CONTAINER  USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
AND
foo root        2409  0.0  0.1 116220 1712 ?        Ss   18:16   0:00 /sbin/init
foo root        2433  0.0  0.0  4060   540 pts/5    Ss+  18:16   0:00 /sbin/minge
tty console
```

```
foo root      2669  0.0  0.1 189880  1532 ?          Sl  18:16  0:00 /sbin/rsysl
ogd -i /var/run/syslogd.pid -c 5
foo dbus      2707  0.0  0.0  21400   908 ?          Ss  18:16  0:00 dbus-daemon
--system
foo 68        2741  0.0  0.2  24928  2360 ?          Ss  18:16  0:00 hald
foo root      2742  0.0  0.1  18104  1120 ?          S   18:16  0:00 hald-runner
foo root      2797  0.0  0.1  66252  1208 ?          Ss  18:16  0:00 /usr/sbin/s
shd
foo root      2819  0.0  0.0 110176   924 ?          Ss  18:16  0:00 /usr/sbin/a
brtd
foo root      2831  0.0  0.1  70984  1232 ?          Ss  18:16  0:00 nginx: mast
er process /opt/17173/nginx/sbin/nginx -c /opt/17173/nginx/conf/nginx.conf
foo nobody    2834  0.0  0.1  71444  1796 ?          S   18:16  0:00 nginx: work
er process
foo root      2840  0.0  0.1 117248  1268 ?          Ss  18:16  0:00 crond
foo root      2851  0.0  0.0  21492   476 ?          Ss  18:16  0:00 /usr/sbin/a
td
foo root      2864  0.0  0.0   4060   540 pts/1      Ss+  18:16  0:00 /sbin/minge
tty /dev/tty1
foo root      2866  0.0  0.0   4060   536 pts/2      Ss+  18:16  0:00 /sbin/minge
tty /dev/tty2
foo root      2868  0.0  0.0   4060   536 pts/3      Ss+  18:16  0:00 /sbin/minge
tty /dev/tty3
foo root      2870  0.0  0.0   4060   536 pts/4      Ss+  18:16  0:00 /sbin/minge
tty /dev/tty4
```

```
# lxc-ps --lxc -- aux          (如果启动着多个实例，就会把所有实例都打印出来)
CONTAINER  USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMM
AND
foo root    2409  0.0  0.1 116220  1712 ?          Ss  18:16  0:00 /sbin/init
foo root    2433  0.0  0.0   4060   540 pts/5      Ss+  18:16  0:00 /sbin/minge
tty console
foo root    2669  0.0  0.1 189880  1532 ?          Sl  18:16  0:00 /sbin/rsysl
ogd -i /var/run/syslogd.pid -c 5
foo dbus    2707  0.0  0.0  21400   908 ?          Ss  18:16  0:00 dbus-daemon
--system
foo 68      2741  0.0  0.2  24928  2360 ?          Ss  18:16  0:00 hald
foo root    2742  0.0  0.1  18104  1120 ?          S   18:16  0:00 hald-runner
foo root    2797  0.0  0.1  66252  1208 ?          Ss  18:16  0:00 /usr/sbin/s
shd
foo root    2819  0.0  0.0 110176   924 ?          Ss  18:16  0:00 /usr/sbin/a
brtd
foo root    2831  0.0  0.1  70984  1232 ?          Ss  18:16  0:00 nginx: mast
er process /opt/17173/nginx/sbin/nginx -c /opt/17173/nginx/conf/nginx.conf
foo nobody  2834  0.0  0.1  71444  1796 ?          S   18:16  0:00 nginx: work
er process
foo root    2840  0.0  0.1 117248  1268 ?          Ss  18:16  0:00 crond
```



```
foo root      2851  0.0  0.0  21492  476 ?          Ss   18:16   0:00 /usr/sbin/a
td
foo root      2864  0.0  0.0   4060   540 pts/1      Ss+  18:16   0:00 /sbin/minge
tty /dev/tty1
foo root      2866  0.0  0.0   4060   536 pts/2      Ss+  18:16   0:00 /sbin/minge
tty /dev/tty2
foo root      2868  0.0  0.0   4060   536 pts/3      Ss+  18:16   0:00 /sbin/minge
tty /dev/tty3
foo root      2870  0.0  0.0   4060   536 pts/4      Ss+  18:16   0:00 /sbin/minge
tty /dev/tty4
```

2.6.8. 实例状态监控

- `lxc-wait` 只支持单一实例，可用于判断

```
# lxc-wait -n foo -s STOPPED 当状态变为STOPPED时程序退出
```

- `lxc-monitor` 高级模式，支持正则

```
# lxc-monitor -n foo
'foo' changed state to [STOPPING]
'foo' changed state to [STOPPED]
'foo' changed state to [STARTING]
'foo' changed state to [RUNNING]
```

2.6.9. LXC操作指令详解

- `lxc`操作指令集

<code>lxc-attach</code>	<code>lxc-checkpoint</code>	<code>lxc-create</code>	<code>lxc-freeze</code>	<code>lxc-ls</code>
<code>lxc-ps</code>	<code>lxc-setuid</code>	<code>lxc-unfreeze</code>	<code>lxc-wait</code>	
<code>lxc-cgroup</code>	<code>lxc-clone</code>	<code>lxc-destroy</code>	<code>lxc-info</code>	<code>lxc-monitor</code>
<code>lxc-restart</code>	<code>lxc-start</code>	<code>lxc-unshare</code>		
<code>lxc-checkconfig</code>	<code>lxc-console</code>	<code>lxc-execute</code>	<code>lxc-kill</code>	<code>lxc-netstat</code>
<code>lxc-setcap</code>	<code>lxc-stop</code>	<code>lxc-version</code>		

- `lxc-attach` (binary)

1. 作用：待补充
2. 用法示例：待补充
3. 注意：待补充

- **lxc-cgroup** (binary)

1. 作用：资源额度配置
2. 用法示例：
查看：`lxc-cgroup -n foo cpuset.cpus`
配置：`lxc-cgroup -n foo cpuset.cpus "0,3"`
3. 注意：只能修改已启动实例的配置，若需要长久生效，需要相应更改配置文件`config`

- **lxc-checkconfig** (bash)

1. 作用：检测内核是否支持
2. 原理：默认读取`/proc/config.gz`，若不存在，则读取`/boot/`uname -r``，进行查找，如是否支持Namespace、Cgroup等
3. 用法示例：
自动查找：`lxc-checkconfig`
手动指定：`CONFIG=/boot/config-2.6.32-358.18.1.el6.x86_64 lxc-checkconfig`
4. 注意：待补充

- **lxc-checkpoint** (binary)

1. 作用：待补充
2. 用法示例：待补充
3. 注意：not implemented yet

- **lxc-clone** (bash)

1. 作用：克隆实例,需要使用lvm
2. 原理：拷贝`config`、`fstab`、`rootfs`，并修改一些信息，如`hostname`等。支持通过lvm实现快照
3. 用法示例：待补充
4. 注意：待补充

- **lxc-console (binary)**

1. 作用：连接实例终端 (tty)
2. 用法示例：`lxc-console -n foo`
3. 注意：lxc-console执行时会创建并连接/var/lib/lxc/command (socket, 不过该socket不是存在于文件系统上, 而是内存里), 可通过netstat -xa查看

- **lxc-create (bash)**

1. 作用：创建实例
2. 原理：从模板或指定位置拷贝config, 通过template脚本进行一些初始化操作 (如ssh模板用于修改/etc/ssh/sshd_config, 做些比较常见的系统初始化操作, mkdir系统根下的一些目录以及写入config默认配置)
3. 用法示例：待补充
4. 注意：待补充

- **lxc-destroy (bash)**

1. 作用：删除实例
2. 原理：`rm -rf /var/lib/lxc/foo`
3. 用法示例：待补充
4. 注意：待补充

- **lxc-execute (binary)**

1. 作用：仅启动容器, 可用于进程资源限制, 与lxc-start不同, 具体作用尚不了解
2. 用法示例：`lxc-execute -n foo -- /bin/bash`
3. 注意：待补充

- **lxc-freeze (binary)**

1. 作用：冻结实例, 等于给容器里所有进程发送SIGSTOP
2. 用法示例：`lxc-freeze -n foo`
3. 注意：待补充

- **lxc-info (binary)**

1. 作用：查看实例状态及其init进程pid
2. 用法示例：
查看状态和pid: `lxc-info -n foo`
只查看状态: `lxc-info -n foo -s`
只查看pid: `lxc-info -n foo -p`
3. 注意：待补充

- **lxc-kill (binary)**

1. 作用：给容器进程（或者init进程，不太了解）发送信号，若不指定信号，则容器会被关闭，相当于`lxc-stop`
2. 用法示例：
指定信号: `lxc-kill -n foo SIGNAL`
不指定信号: `lxc-kill -n foo` 经实验发现作用和`lxc-stop -n foo`一样
3. 注意：待补充

- **lxc-ls (bash)**

1. 作用：列出所有实例，包括运行和未运行
2. 原理：`ls /var/lib/lxc`以及`netstat -xa | grep '/var/lib/lxc'`，`lxc-start`、`lxc-console`均会创建到`/var/lib/lxc/foo/command`套接字的连接，且无法改变位置，因此无论lxc实例放在哪，只要启动了，`lxc-ls`都能显示。
3. 用法示例：
简单输出: `lxc-ls`
详细输出: `lxc-ls -l`
4. 注意：
 - 1) `lxc-ls`命令能使用的前提是存在`/var/lib/lxc`目录
 - 2) 若当前有`lxc-console`连接着终端，那么`lxc-ls`也会显示，且不做`uniq`过滤，因此可能出现多个相同名字的实例。这点很不靠谱
 - 3) 建议将lxc实例放在`/opt/lxc/`目录下，且保持`/var/lib/lxc`目录存在且为空！

- **lxc-monitor (binary)**

1. 作用：实例状态监控（仅支持屏幕输出）
2. 用法示例: `lxc-monitor -n 正则`（支持监控多个实例）
`lxc-monitor -n foo`
`lxc-monitor -n 'foo|bar'`

```
lxc-monitor -n '[f|b].*'
lxc-monitor -n '.*'
```

3. 注意:

- 1) 容器状态实时输出到屏幕上，有输出到日志的选项，但没有效果，依旧只输出到屏幕上，使用>导入文件也不行，猜测不是实时写入，而是先缓存到内存里，导致无法从文件调取，这点很不靠谱
- 2) 会与lxc-wait冲突，只能使用其一

• lxc-netstat (bash)

1. 作用: 查看实例当前网络连接
2. 原理: `mount --bind /proc/实例init的pid/net /proc/$$/net && exec netstat "$@"`
3. 用法示例: `lxc-netstat -n foo lntpu`
4. 注意: 待补充

• lxc-ps (perl)

1. 作用: 查看实例当前进程报告
2. 原理: 通过/`cgroup/foo/tasks`获取实例的所有进程pid，然后直接从宿主机进程报告里筛选出来，因为实例本身就是宿主机的进程
3. 用法示例:
查看单个实例: `lxc-ps -n foo -- aux` 相当于登录进实例执行了`ps aux`
查看所有实例: `lxc-ps --lxc -- -ef` 相当于登录进所有lxc实例的终端，执行了`ps -ef`
4. 注意: 虽然lxc-ps --lxc选项也是通过netstat -xa实现，但可能后期经过类似uniq处理，因此lxc-ps的输出是正确的，不会像lxc-ls那样输出相同的

• lxc-restart (binary)

1. 作用: 待补充
2. 用法示例: 待补充
3. 注意: not implemented yet

• lxc-setcap (bash)

1. 作用: 使用libcap对lxc操作指令集进行cap设置 (什么是cap: <http://www.cnblogs.com/iamfy/archive/2012/09/20/2694977.html>)
2. 用法示例: 待补充
3. 注意: 待补充

- **lxc-setuid (bash)**

1. 作用：添加或删除lxc操作指令集的setuid位
2. 用法示例：
添加s位： `lxc-setuid`
删除s位： `lxc-setuid -d`
3. 注意： `lxc-setuid`会创建 `/var/lib/lxc` 目录并赋777权限，执行 `lxc-setuid -d` 会赋 `/var/lib/lxc` 目录755权限

- **lxc-start (binary)**

1. 作用：启动实例
2. 用法示例：
调试使用： `lxc-start -n foo -f /opt/lxc/foo/config -o /opt/lxc/foo/foo.log -l debug`
daemon模式： `lxc-start -n foo -f /opt/lxc/foo/config -o /opt/lxc/foo/foo.log -l debug -d`
3. 注意：一定不能用 `kill -9` 来杀 `lxc-start` 进程，因为这样 `/cgroup` 里的 `name` 目录就去不掉了，只能重启宿主机

- **lxc-stop (binary)**

1. 作用：关闭实例
2. 用法示例： `lxc-stop -n foo`
3. 注意：待补充

- **lxc-unfreeze (binary)**

1. 作用：解冻实例
2. 用法示例：待补充
3. 注意：待补充

- **lxc-unshare (binary)**

1. 作用：待补充

2. 用法示例：待补充
3. 注意：待补充

- **lxc-version (bash)**

1. 作用：查看lxc版本
2. 原理：脚本就一行`echo "lxc version: 0.7.5"`
3. 用法示例：还用说吗
4. 注意：待补充

- **lxc-wait (binary)**

1. 作用：等待状态。可用于简单状态监测，如执行`lxc-stop`前先`lxc-wait STOPPED`，当`lxc-wait`执行完毕退出即表示`lxc-stop`执行成功。
2. 用法示例：
`lxc-wait -n foo -s RUNNING` 当foo的状态遇到RUNNING时结束退出
`lxc-wait -n foo -s 'RUNNING|STOPPED'` 当foo的状态遇到RUNNING或STOPPED时结束退出
3. 注意：
 - 1) 该命令与`lxc-monitor`冲突，只能用一个命令，无法两者一起使用。
 - 2) 在实例里执行`reboot`，也会触发状态。

2.7. 网络类型

2.7.1. 物理（**phys**）网络

- 指定一个物理接口作为容器网络使用：

```
lxc.network.type = phys
lxc.network.flags = up
lxc.network.link = eth0
lxc.network.hwaddr = 4a:49:43:49:79:ff
lxc.network.ipv4 = 10.10.95.87/24
lxc.network.ipv6 = 2003:db8:1:0:214:1234:fe0b:3297
```

- 优点：配置简单
- 缺点：网卡资源有限

2.7.2. 配置桥接（veth）网络

- **veth (virtual ethernet)**宿主需要开启桥接模式，配置如下：添加桥接网卡

```
# vim ifcfg-br0
DEVICE="br0"
ONBOOT="yes"
TYPE="Bridge"
BOOTPROTO="static"
IPADDR="192.168.108.176"
NETMASK="255.255.252.0"
GATEWAY="192.168.111.254"
```

修改eth0网卡

```
# vim ifcfg-eth0
DEVICE="eth0"
BRIDGE="br0"
ONBOOT="yes"
BOOTPROTO="none"
NM_CONTROLLED="no"
TYPE="Ethernet"
HWADDR="00:16:3E:47:3E:D4"
```

Lxc的配置文件格式如下：

```
lxc.network.type = veth
lxc.network.flags = up
lxc.network.link=br0
lxc.network.veth.pair= vethxxx（最多15个字符）
lxc.network.hwaddr= 00:16:3e:77:52:20
lxc.network.ipv4 = 10.11.6.225/24
lxc.network.name = eth0
```

启动后宿主会增加一个vethxxx的网卡，容器通过这个网卡连接到宿主的网桥上通信。

2.7.3. 配置macvlan网络

- **macvlan (mac-address based virtual lan tagging)**

- **VEPA (Virtual Ethernet Port Aggregator)**

2.7.3.1. Macvlan分为三种类型:

- **VEPA**模式是默认模式, 这种模式只能一端到另一端连接, 所以容器只能和宿主连接, 容器彼此不能相互访问。
- **Bridge**模式, 所有容器通过mac地址标记, 彼此可以直接通信, 由于kernel知道每个mac地址的分配, 所以实际上容器之间是通过内核通信。Mac地址可以随便定义, 但是ip地址必须符合规则, 必须在同一网段的。
- **Private**模式, 这种模式下的容器不能与任何其他机器通讯。

2.7.3.2. 宿主机MAC Bridge配置

- 宿主配置虚拟网桥:

```
ip link add link eth0 name macv0 address 00:aa:bb:cc:dd:10 type macvlan mode bridge
ip link set macv0 up
ip address add 172.16.100.10/24 broadcast 172.16.100.255 dev macv0
```

- 容器一:

```
lxc.network.type = macvlan
lxc.network.flags = up
lxc.network.link = macv0
lxc.network.macvlan.mode = bridge
lxc.network.hwaddr= 00:aa:bb:cc:dd:11
lxc.network.ipv4 = 172.16.100.11/24
```

- 容器二:

```
lxc.network.type = macvlan
lxc.network.flags = up
lxc.network.link = macv0
lxc.network.macvlan.mode = bridge
lxc.network.hwaddr= 00:aa:bb:cc:dd:12
lxc.network.ipv4 = 172.16.100.12/24
```

- 查看网桥日志

```
# ip -s link show macv0
```

```
66: macv0@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state U
NKNOWN
```

```
link/ether 00:aa:bb:cc:dd:10 brd ff:ff:ff:ff:ff:ff
```

```
RX: bytes  packets  errors  dropped  overrun  mcast
```

```
19608      264      0        0        0        261
```

```
TX: bytes  packets  errors  dropped  carrier  collsns
```

```
790        11        0        0        0        0
```

- 删除虚拟网桥。

```
# ip link delete macv0
```

- **macvlan**采用**bridge**模式, 每个容器**link**到同一块网卡**macv0**上(可以是物理网卡, 也可以是容器网卡), 容器指定自己独立的**MAC**和**IP**地址, **IP**地址可以根据**vlan**分配, 实际上两个容器通过**IP**映射到**MAC**地址上, 然后相互访问。
- **MacVlan**内的容器要想访问外网其他段的地址, 需要把容器的网关配置为网桥的地址。宿主用**iptables**做**nat**伪装。

```
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

- 外面的服务器容器的**ip**地址, 实际上是访问宿主的**eth0**网卡, 然后通过判断将请求转到不同的容器上。

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT --to-destination
11.11.11.11:80
```

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 81 -j DNAT --to-destination
11.11.11.12:80
```

- 也可以使用**socat**方式转发, **socat**(Socket Cat)是**NC**(Net Cat)的增强版

```
socat -d -d -lf /var/log/socat.log TCP4-LISTEN:80,reuseaddr,fork,su=nobody TCP4:
11.11.11.11:80 &
```

2.7.3.3. 配置vlan网络

- 每个vlan.id就是一个网段，每个网卡上的vlan.id不能重复。这样就相当于容器是一个网关服务器，隔离各个局域网。
- 配置举例：

```
lxc.network.type = vlan
lxc.network.vlan.id = 100
lxc.network.flags = up
lxc.network.link = eth0
lxc.network.ipv4 = 202.202.0.25/24
```

- 上述配置vlan方法没具体测试过，推荐方法：宿主机添加vlan及对应桥接网卡需求：宿主机的网段是67，现需要在上面运行lxc容器，但是lxc容器的网段是74

步骤简述：

- 一．添加宿主网卡当前vlan
- 二．添加宿主对应1的vlan的桥接网卡
- 三．添加宿主网卡新vlan
- 四．添加宿主对应3的vlan的桥接网卡
- 五．修改容器config的桥接网卡对象

假设宿主网卡是em2，桥接网卡是br2，那么：

- 一．添加网卡vlan

```
vim ifcfg-em2.67
DEVICE="em2.67"
BRIDGE="br2"
BOOTPROTO="none"
NM_CONTROLLED="no"
ONBOOT="yes"
TYPE="Ethernet"
VLAN=yes
```

注：.67表示vlan id为67,这个vlan为宿主机管理ip的vlan

- 二．将一的网卡桥接到br2上

```
vim ifcfg-br2
DEVICE="br2"
TYPE="Bridge"
NM_CONTROLLED="no"
ONBOOT="yes"
BOOTPROTO="none"
IPADDR=192.168.1.1
```

```
NETMASK=255.255.255.0
```

三. 添加新段网卡vlan

```
vim ifcfg-em2.74
DEVICE="em2.74"
BRIDGE="br2.74"
BOOTPROTO=none
NM_CONTROLLED="no"
ONBOOT="yes"
TYPE="Ethernet"
VLAN=yes
```

四. 将三的网卡桥接到br2.74

```
DEVICE="br2.74"
TYPE="Bridge"
NM_CONTROLLED="no"
ONBOOT="yes"
BOOTPROTO="none"
```

五. 容器桥接到br2.74上

```
vim config (仅截出em2网络部分)
lxc.network.type = veth
lxc.network.flags = up
lxc.network.link = br2.74
lxc.network.name = em2
lxc.network.veth.pair = cctest-em2
lxc.network.ipv4 = 192.168.2.1/24
```

下面为完整的相关配置信息截取:

```
=====
```

```
[ 16:52:02-root@J8YF43X:lxc ]#ifconfig
```

```
br1      Link encap:Ethernet  HWaddr 78:2B:CB:32:15:94
          inet6 addr: fe80::7a2b:cbff:fe32:1594/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:6510 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:379712 (370.8 KiB)  TX bytes:468 (468.0 b)

br2      Link encap:Ethernet  HWaddr 78:2B:CB:32:15:95
          inet addr:192.168.2.1  Bcast:192.168.2.255  Mask:255.255.255.0
          inet6 addr: fe80::7a2b:cbff:fe32:1595/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:892701 errors:0 dropped:0 overruns:0 frame:0
          TX packets:689009 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:1380281321 (1.2 GiB)  TX bytes:49896535 (47.5 MiB)

br2.74   Link encap:Ethernet  HWaddr 78:2B:CB:32:15:95
          inet6 addr: fe80::7a2b:cbff:fe32:1595/64 Scope:Link
```

```
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:173 errors:0 dropped:0 overruns:0 frame:0
TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:12536 (12.2 KiB)  TX bytes:468 (468.0 b)

cctest-em1 Link encap:Ethernet  HWaddr 0A:09:40:E3:7E:8D
        inet6 addr: fe80::809:40ff:fee3:7e8d/64 Scope:Link
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:4 errors:0 dropped:0 overruns:0 frame:0
TX packets:25 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:384 (384.0 b)  TX bytes:2198 (2.1 KiB)

cctest-em2 Link encap:Ethernet  HWaddr 9E:CD:18:E3:EC:FA
        inet6 addr: fe80::9ccd:18ff:fee3:ecfa/64 Scope:Link
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:6 errors:0 dropped:0 overruns:0 frame:0
TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:496 (496.0 b)  TX bytes:544 (544.0 b)

em1      Link encap:Ethernet  HWaddr 78:2B:CB:32:15:94
        inet6 addr: fe80::7a2b:cbff:fe32:1594/64 Scope:Link
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:6568 errors:0 dropped:0 overruns:0 frame:0
TX packets:53 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:514104 (502.0 KiB)  TX bytes:4090 (3.9 KiB)

em2      Link encap:Ethernet  HWaddr 78:2B:CB:32:15:95
        inet6 addr: fe80::7a2b:cbff:fe32:1595/64 Scope:Link
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:1207560 errors:0 dropped:0 overruns:0 frame:0
TX packets:559955 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:1381625253 (1.2 GiB)  TX bytes:46747231 (44.5 MiB)

em2.67   Link encap:Ethernet  HWaddr 78:2B:CB:32:15:95
        inet6 addr: fe80::7a2b:cbff:fe32:1595/64 Scope:Link
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:672744 errors:0 dropped:0 overruns:0 frame:0
TX packets:547185 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:1326660440 (1.2 GiB)  TX bytes:38035951 (36.2 MiB)

em2.74   Link encap:Ethernet  HWaddr 78:2B:CB:32:15:95
        inet6 addr: fe80::7a2b:cbff:fe32:1595/64 Scope:Link
UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
RX packets:1097 errors:0 dropped:0 overruns:0 frame:0
```

```
TX packets:688 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:87287 (85.2 KiB) TX bytes:115723 (113.0 KiB)

lo      Link encap:Local Loopback
        inet addr:127.0.0.1  Mask:255.0.0.0
        inet6 addr: ::1/128 Scope:Host
        UP LOOPBACK RUNNING  MTU:16436  Metric:1
        RX packets:29427 errors:0 dropped:0 overruns:0 frame:0
        TX packets:29427 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:22590372 (21.5 MiB) TX bytes:22590372 (21.5 MiB)

[ 16:52:04-root@J8YF43X:lxc ]#ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: em1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP qlen 1000
    link/ether 78:2b:cb:32:15:94 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::7a2b:cbff:fe32:1594/64 scope link
        valid_lft forever preferred_lft forever
3: em2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP qlen 1000
    link/ether 78:2b:cb:32:15:95 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::7a2b:cbff:fe32:1595/64 scope link
        valid_lft forever preferred_lft forever
737: br2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN
    link/ether 78:2b:cb:32:15:95 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.1/24 brd 192.168.2.255 scope global br2
    inet6 fe80::7a2b:cbff:fe32:1595/64 scope link
        valid_lft forever preferred_lft forever
789: br1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN
    link/ether 78:2b:cb:32:15:94 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::7a2b:cbff:fe32:1594/64 scope link
        valid_lft forever preferred_lft forever
790: em2.67@em2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
UP
    link/ether 78:2b:cb:32:15:95 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::7a2b:cbff:fe32:1595/64 scope link
        valid_lft forever preferred_lft forever
791: em2.74@em2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
UP
    link/ether 78:2b:cb:32:15:95 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::7a2b:cbff:fe32:1595/64 scope link
        valid_lft forever preferred_lft forever
792: br2.74: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKN
```

```
OWN
    link/ether 78:2b:cb:32:15:95 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::7a2b:cbff:fe32:1595/64 scope link
        valid_lft forever preferred_lft forever
819: cctest-em2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast sta
te UP qlen 1000
    link/ether 9e:cd:18:e3:ec:fa brd ff:ff:ff:ff:ff:ff
    inet6 fe80::9ccd:18ff:fee3:ecfa/64 scope link
        valid_lft forever preferred_lft forever
821: cctest-em1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast sta
te UP qlen 1000
    link/ether 0a:09:40:e3:7e:8d brd ff:ff:ff:ff:ff:ff
    inet6 fe80::809:40ff:fee3:7e8d/64 scope link
        valid_lft forever preferred_lft forever

[ 16:52:16-root@J8YF43X:lxc ]#brctl show
bridge name      bridge id                STP enabled    interfaces
br1               8000.782bcb321594        no             cctest-em1
                                                         em1
br2               8000.782bcb321595        no             em2
                                                         em2.67
br2.74            8000.782bcb321595        no             cctest-em2
                                                         em2.74

[ 16:52:35-root@J8YF43X:network-scripts ]#ls
backup-20140408-143253  ifcfg-br2.74  ifcfg-em2.67  ifdown        ifdown-ipp   if
down-post              ifdown-sit    ifup-aliases  ifup-ipp      ifup-plip    ifup-ppp     if
up-tunnel               net.hotplug   route-br2
ifcfg-br1               ifcfg-em1     ifcfg-em2.74  ifdown-bnep   ifdown-ipv6  if
down-ppp                ifdown-tunnel ifup-bnep      ifup-ipv6     ifup-plusb   ifup-routes  if
up-wireless             network-functions
ifcfg-br2               ifcfg-em2     ifcfg-lo       ifdown-eth    ifdown-isdn  if
down-routes             ifup          ifup-eth       ifup-isdn     ifup-post    ifup-sit     in
it.ipv6-global          network-functions-ipv6

[ 16:52:36-root@J8YF43X:network-scripts ]#cat ifcfg-br1
DEVICE="br1"
TYPE="Bridge"
NM_CONTROLLED="no"
ONBOOT="yes"
BOOTPROTO="none"

[ 16:52:38-root@J8YF43X:network-scripts ]#cat ifcfg-br2
DEVICE="br2"
TYPE="Bridge"
NM_CONTROLLED="no"
ONBOOT="yes"
BOOTPROTO="none"
IPADDR=192.168.2.1
NETMASK=255.255.255.0

[ 16:52:39-root@J8YF43X:network-scripts ]#cat ifcfg-br2.74
```

```
DEVICE="br2.74"
TYPE="Bridge"
NM_CONTROLLED="no"
ONBOOT="yes"
BOOTPROTO="none"
[ 16:52:42-root@J8YF43X:network-scripts ]#cat ifcfg-em1
DEVICE="em1"
BRIDGE="br1"
BOOTPROTO="none"
NM_CONTROLLED="no"
ONBOOT="yes"
TYPE="Ethernet"
[ 16:52:44-root@J8YF43X:network-scripts ]#cat ifcfg-em2
DEVICE="em2"
BOOTPROTO="none"
NM_CONTROLLED="no"
ONBOOT="yes"
TYPE="Ethernet"
[ 16:52:45-root@J8YF43X:network-scripts ]#cat ifcfg-em2.67
DEVICE="em2.67"
BRIDGE="br2"
BOOTPROTO="none"
NM_CONTROLLED="no"
ONBOOT="yes"
TYPE="Ethernet"
VLAN=yes
[ 16:52:48-root@J8YF43X:network-scripts ]#cat ifcfg-em2.74
DEVICE="em2.74"
BRIDGE="br2.74"
BOOTPROTO=none
NM_CONTROLLED="no"
ONBOOT="yes"
TYPE="Ethernet"
VLAN=yes

[ 16:53:04-root@J8YF43X:cctest ]#cat config
## NETWORK- ##
## em2- ##
lxc.network.type = veth
lxc.network.flags = up
lxc.network.link = br2.74
lxc.network.name = em2
lxc.network.veth.pair = cctest-em2
lxc.network.ipv4 = 192.168.2.10/24
## -em2 ##
## -NETWORK ##

## NETWORK- ##
## em1- ##
```



```
lxc.network.type = veth
lxc.network.flags = up
lxc.network.link = br1
lxc.network.name = em1
lxc.network.veth.pair = cctest-em1
## -em1 ##
## -NETWORK ##
=====
```

4. 神器

4.1. nsenter（无需ssh、无需attach也可以登录容器）

- 原理：进入namespace（通过/proc/xxxx/ns/）
- 安装：

```
wget https://www.kernel.org/pub/linux/utils/util-linux/v2.25/util-linux-2.25.tar
.gz
解压，进入目录
./configure --without-ncurses
make LDFLAGS=-all-static nsenter
cp nsenter /usr/local/bin/
```

- 使用：nsenter -target \$PID -mount -uts -ipc -net -pid # 这个\$PID指容器里的任意进程在宿主上的真实PID

5. FAQ

5.1. 容器启动报错

5.1.1. Too many open files – failed to inotify_init

- 宿主修改/etc/sysctl.conf, 增加:

```
fs.inotify.max_user_instances = 2048
```

5.2. resin/httpd等服务非正常关闭, 导致容器出错

- 错误现象:

```
# 在容器中reboot, 发现如下报错:
lxc-start: failed to create vnet-bj-test-12-vethaxnYbe : File exists
lxc-start: failed to create netdev
lxc-start: failed to create the network
lxc-start: failed to spawn 'bj-test-12'
lxc-start: No such file or directory - failed to remove cgroup '/cgroup/bj-test-12'

# 发现容器已经关闭, 但虚拟网卡仍然存在:

# ifconfig
vnet-bj-test-12 Link encap:Ethernet  HWaddr 76:B3:A7:30:D0:47
    inet6 addr: fe80::74b3:a7ff:fe30:d047/64 Scope:Link
    UP BROADCAST MULTICAST  MTU:1500  Metric:1
    RX packets:1701239 errors:0 dropped:0 overruns:0 frame:0
    TX packets:379753 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:93329189 (89.0 MiB)  TX bytes:4487435319 (4.1 GiB)

# 强行关闭改网卡后, 容器恢复正常:

# ip link delete vnet-bj-test-12
```

- 导致这种状态的原因:

在容器中发现启动了`resin/httpd`等内存、监听器回收较慢的服务, 猜测`lxc`关闭过程太快, 这些程序使用的宿主网卡资源未释放, 导致这个问题产生, 删除`resin`自启动服务后现象消失。解决办法: 先将容器的应用进程都杀掉, 等到`netstat -antup`查看没有连接的进程时, 再重启或关闭容器

5.3. 容器名被占用的解决办法

当直接将`lxc-start`后台进程`kill -9`或者容器资源消耗太大导致使用`reboot/poweroff`或宿主上使用`lxc-stop`仅关闭了`lxc-start`进程, 但`/cgroup`下的命名空间(即容器名)并未消失, 则无法使用相同的容器名启动容器, 解决办法有两种:

5.3.1. 更换容器名(不建议)

- 不使用原有的容器名, 而更换个容器名来启动容器, 不过这样存在一个问题, 就是容器里的部分进程可能还存在, 导致资源未全部释放

5.3.2. 强制删除

- 第一步：安装libcgroup

```
yum install libcgroup
```

安装完会在/etc/init.d/下生成2个服务：cgconfig, cgred。这2个服务默认是关着的，且开机不启动的。

此时应该有了cgdelete指令

- 第二步：杀进程

```
cd /cgroup/name/ #name表示命名空间，对lxc来说就是容器名
for i in `cat cgroup.procs`; do
    kill -9 $i
done
```

```
cat cgroup.procs #查看下是否还有进程残留，若有，继续杀
```

注意这里kill的进程是来自cgroup.procs而不是tasks的。cgroup.procs与tasks的区别：cgroup.procs是进程ID，而tasks是线程ID

- 第三步：使用cgdelete工具删掉命名空间

```
cgdelete 任一子系统:/name
```

如cgdelete cpu:/v-bj-test-11或cgdelete blkio:/v-bj-test-11均可

还有一个更简单的方法：cd /cgroup; rmdir v-bj-test-11（注意rm -rf是不行的，只能用rmdir）

5.4. LXC宿主机上做LVS时，真实服务器不能是同一台宿主机下面的LXC容器

5.5. 网卡名长度限制

- 实验发现lxc.network.veth.pair参数有长度限制

最多为15个字符，不能超过15个字符，测试支持的字符有数字、字母，还有3个符号-_.
其他符号没尝试过

`lxc.utsname`和`name`测试到50个字符依然没有出现问题（`name`指容器标识符，如`lxc-start -n name`）

5.6. 执行dmidecode报错”/dev/mem: No such file or directory”

- 在容器里需要执行以下命令，并在`config`里加入`lxc.cgroup.devices.allow = c 1:1 rwm`

```
mknod -m 640 /dev/mem c 1 1
chown root:kmem /dev/mem
```

虽然执行`dmidecode`不会报错了，但输出的内容实际是宿主机的

5.7. 加载fuse报错

- 使用MFS等需要加载`fuse`才可使用的文件系统时，需要创建字符设备文件及`cgroup`授权才可使用：

在容器里执行：`mknod -m 660 /dev/fuse c 10 229`

在宿主上进行`cgroup device`授权：

1. 永久生效：在`config`里添加`lxc.cgroup.devices.allow = c 10:229 rwm`

2. 即时生效：`echo c 10:229 rwm > /cgroup/name/devices.allow`

取消：`echo c 10:229 rwm > /cgroup/name/devices.deny`

查看已生效的：`cat /cgroup/name/devices.list`

第一次测试MFS发现：

若提示需要`modprobe fuse`，需要在宿主机上执行该命令（当`drop CAP_SYS_MODULE`时，即配置了`lxc.cap.drop = sys_module`，则在容器里无法加载和卸载模块，如果允许的话会影响宿主机，因为LXC是共享内核空间，并非独立的内核空间）

第二次测试MFS发现：

`mfsmount`时候会自动加载`fuse`，而且居然成功了，从宿主上`lsmod fuse`也能看到被加载了！

支持的设备类型：`a` 应用所有设备，可以是字符设备，也可以是块设备；`b` 指定块设备；`c` 指定字符设备

关于`rwm`的解释：`r` 允许任务从指定设备中读取；`w` 允许任务写入指定设备；`m` 允许任务生成还不存在的设备文件

- `mknod`系统设备小知识
 - `/dev/`下块设备和字符设备的含义

```
mknod DEVNAME {b | c} MAJOR MINOR
```

`b`和`c` 分别表示块设备和字符设备：

`b`：表示系统从块设备中读取数据的时候，直接从内存的`buffer`中读取数据，而不经磁盘；

`c`：表示字符设备文件与设备传送数据的时候是以字符的形式传送，一次传送一个字符，比如打印机、终端都是以字符的形式传送数据；

`MAJOR`和`MINOR`分别表示主设备号和次设备号：

为了管理设备，系统为每个设备分配一个编号，一个设备号由主设备号和次设备号组成。

主设备号标示某一种类的设备，次设备号用来区分同一类型的设备。

`linux`操作系统中为设备文件编号分配了32位无符号整数，其中前12位是主设备号，后20位为次设备号。

所以在向系统申请设备文件时主设备号不好超过4095，次设备号不好超过 $2^{20} - 1$

- `cgroup`里`devices.allow`添加的号码是登录容器里看到的，而不是宿主文件系统上看到的

5.8. 宿主机问题导致的容器使用异常

5.8.1. TTY问题

- `ssh`宿主失败

```
root@...'s password:
PTY allocation request failed on channel 0
```

- 启动容器报错

```
# lxc-start -n foo
lxc-start: Permission denied - failed to create pty #0
```

```
lxc-start: failed to create the ttys
lxc-start: failed to initialize the container
```

上述”ssh宿主失败”和”启动容器报错”现象均是因为在宿主上无法创建/dev/tty或/dev/pts/X（X表示数字）所致

- 导致问题的原因

当容器中不存在/etc/mtab或/etc/mtab中包含了devpts的挂载信息，在用reboot、poweroff、shutdown、halt等命令关机时会将宿主的devpts重新挂载为ro

- 解决方法

```
1. 确保/dev/ptmx存在，若不存在，则ln -s /dev/pts/ptmx /dev/ptmx
2. 确保devpts挂载为rw（可读可写），可从/proc/mounts里看到，若为ro，则mount -o remount,
   rw devpts（建议写入cron: * * * * * mount -o remount,rw devpts）
```

5.9. 添加tty, 允许更多的终端登录

- 原容器配置文件(config)中, lxc.tty = 1

容器配置文件修改:

```
# cat config | grep tty
lxc.tty = 4
```

容器操作系统中需要创建:

```
mknod -m 660 /dev/tty1 c 4 1
mknod -m 660 /dev/tty2 c 4 2
mknod -m 660 /dev/tty3 c 4 3
mknod -m 660 /dev/tty4 c 4 4
chown root:tty /dev/tty1
chown root:tty /dev/tty2
chown root:tty /dev/tty3
chown root:tty /dev/tty4
```

5.10. 服务检查

- 关闭无用服务

```
# cat service_check.sh
chkconfig --level 35 abrt-ccpp off
chkconfig --level 35 abrt-d off

# 电源的开关等检测管理，常用在Laptop上，关闭
chkconfig --level 35 acpid off

# 使用crond，关闭
chkconfig --level 35 atd off

# LVM block management tool
chkconfig --level 35 blk-availability off

# 调节cpu速度用来省电，常用在Laptop上，关闭
chkconfig --level 35 cpuspeed off
chkconfig --level 35 libvirt-guests off
chkconfig --level 35 rhsmcertd off

# 硬盘自动检测守护进程，关闭
chkconfig --level 35 smartd off
chkconfig --level 35 iptables off
chkconfig --level 35 ip6tables off

# lvm监控，关闭
chkconfig --level 35 lvm2-monitor off
chkconfig --level 35 sendmail off

# 自动加载nfs等网络文件系统，关闭
chkconfig --level 35 netfs off

# 软raid监控，关闭
chkconfig --level 35 mdmonitor off

chkconfig --level 35 auditd off
chkconfig --level 35 autofs off
chkconfig --level 35 certmonger off
chkconfig --level 35 nfslock off
chkconfig --level 35 portreserve off
chkconfig --level 35 postfix off
chkconfig --level 35 rhnsd off
chkconfig --level 35 rpcbind off
chkconfig --level 35 rpcgssd off
chkconfig --level 35 rpcidmapd off
chkconfig --level 35 cups off
chkconfig --level 35 haldaemon off
chkconfig --level 35 messagebus off
```

5.11. UTF中文字符处理

- /etc/sysconfig/i18n设置

```
# cat /etc/sysconfig/i18n
LANG="en_US.UTF-8"
SYSFONT="latarcyrheb-sun16"
```

5.13. 操作系统时区与硬件时钟

- 设置上海时间

```
# cp /usr/share/zoneinfo/Asia/Shanghai /etc/localtime
```

- 查看硬件时钟

```
# hwclock --show
```

- 可读不可写

config里的lxc.cap.drop添加sys_time, 这样容器里可以date查看时间、hwclock --show查看硬件时钟, 但是无法通过date -s或hwclock --set --date来修改系统时间和硬件时钟
否则容器里更改系统时间或硬件时钟会导致宿主机的系统时间或硬件时钟同时发生改变, 将会影响宿主机自身和宿主机上其他的容器

5.14. 预读取/预处理脚本删除

- lxc中各种预读脚本不能使用

```
# cd /etc/init/
# rm readahead*
```

5.15. 使用ssh/scp等出现问题

- ssh root@192.168.3.5 发生错误 “Host key verification failed.”, 解决方法

```
# mknod -m 660 /dev/tty c 5 0
# chown root:tty /dev/tty
```

5.16. 桥接网卡MAC绑定

- 容器启动时，容器mac地址随机生成，若此mac地址小于宿主机br1的mac地址，则宿主机br1的mac地址会变更为容器mac地址，当此容器关闭后，宿主机mac地址变更为其他小于宿主机的容器mac地址或宿主机em1的mac地址，解决方法：

```
在/etc/sysconfig/network-scripts/ifup-eth脚本里的70行左右(if [ "${TYPE}" = "Bridge"
" ]; then .....; fi 段落的fi之前)添加:
# add by 17173-sys 陈承
# 用途：默认情况下桥接网卡MAC地址会动态设置为当前机器网卡的最小MAC
#       因此需要手动绑定MAC地址以防止变更造成的物理主机网络短时中断
ip link set br1 address $(get_hwaddr em1)
```

也可手动执行命令 `ip link set br1 address MAC地址`，但桥接网卡重启或宿主机重启后绑定失效

5.17. lxc-monitor工作异常

- 若实例已启动而再次执行lxc-start命令（虽然不会报错），但因为这样lxc-monitor监听到的实例状态会变为STOPPED，而实际上依然是RUNNING，这应该是个BUG

```
'lxc-test1' changed state to [STARTING]
'lxc-test1' changed state to [STOPPING]
'lxc-test1' changed state to [STOPPED]
```

5.18. OOM问题

- 宿主的OOM与容器的OOM并无关联

在宿主上`vm.overcommit_memory=2`（关闭OOM）或者`vm.overcommit_memory=0`（开启OOM）都不会影响容器的OOM是开启或关闭

- 容器的OOM开启与关闭是由cgroup的子系统参数`memory.oom_control`决定
 - `echo 1 > memory.oom_control`表示关闭OOM，`echo 0 > memory.oom_control`表示开启OOM
 - 可以在config里做永久配置：`lxc.cgroup.memory.oom_control = 0`（表示容器开启OOM）
 - `memory.oom_control`里的`under_oom`表示当前是否处在OOM状态，1表示内存满，0表示内存未到上限（无论容器是否开启OOM：`memory.oom_control`）
- 经过测试：若`memory.swappiness`为0，表示不启用swap，则当内存满了，就触发OOM了，若`memory.swappiness>0`，则当swap也满了才会触发OOM。（不一定准确）

5.19. 修改容器里内核参数是否会影响宿主

- 实验发现，有的内核参数会影响，如`vm.overcommit_memory`：当容器里修改了此值，宿主的该项内核参数也会同时跟着变
- 有的内核参数不会影响，如`net.ipv4.ip_forward`

猜测：`namespace`的参数不会受影响，如（网络、`pid`、`ipc`、`mnt`、`uts`），其他的尚待实践考证

- 实验总结：

把kickstart装完后`/etc/sysctl.conf`里的都测了一遍，发现有3种结果：

当`/etc/sysctl.conf`里对参数没有显式设置或者不存在`/etc/sysctl.conf`时

以下3个参数会自动参照宿主

```
net.ipv4.ip_forward
net.ipv4.conf.default.rp_filter
net.ipv4.conf.default.accept_source_route
```

以下4个会自动设置为

```
kernel.msgmnb = 16384
kernel.msgmax = 8192
kernel.shmmax = 33554432
kernel.shmall = 2097152
```

会影响宿主的参数：

```
net.ipv4.tcp_syncookies
net.bridge.bridge-nf-call-ip6tables
net.bridge.bridge-nf-call-iptables
net.bridge.bridge-nf-call-arptables
net.ipv4.tcp_max_syn_backlog
```

```
net.ipv4.tcp_synack_retries
net.ipv4.tcp_syn_retries
net.ipv4.tcp_fin_timeout
net.ipv4.tcp_keepalive_time
net.ipv4.tcp_max_orphans
net.ipv4.tcp_mem
net.ipv4.ip_local_port_range
net.ipv4.tcp_rmem
net.ipv4.tcp_wmem
net.ipv4.tcp_no_metrics_save
net.ipv4.tcp_retrans_collapse
net.ipv4.tcp_window_scaling
net.ipv4.tcp_tw_recycle
net.ipv4.tcp_tw_reuse
net.ipv4.tcp_timestamps
```

当`sysctl.conf`里显式设置的内核参数都会生效

5.20. Capabilities（Linux能力）

- lxc-0.7.5的capabilities支持34个，和man capabilities里的相同，记录在lxc-0.7.5源码包里的conf.c

5.21. 容器已启动，但**console**终端控制台空白无显示

- 原因：容器里找不到对应的/sbin/mingetty /dev/ttyX进程
- 解决方法：在容器里手动执行命令

```
initctl start tty TTY=/dev/tty1
initctl start tty TTY=/dev/tty2
initctl start tty TTY=/dev/tty3
initctl start tty TTY=/dev/tty4
initctl start tty TTY=/dev/tty5
initctl start tty TTY=/dev/tty6
```

5.23. 容器里使用**nfs**挂载

- 需要安装几个包：

```
yum install nfs-utils.x86_64 nfs-utils-lib.x86_64 nfs4-acl-tools.x86_64
```

抓包测试发现数据包源ip是宿主ip而不是容器ip，因此nfs server的/etc/exports里要给宿主ip开权限，而不用给容器ip开权限。原因暂时未知

5.24. 容器充当nfs server

- 怀疑由于红帽6不支持nfs namespace，因此无法充当nfs server
- 解决办法：利用User-space NFSv3 Server（unfsd）实现LXC容器充当nfs server

官网：<http://unfs3.sourceforge.net/>

注：已在RHEL6.4 x86_64下实验成功

LXC宿主：

```
/etc/init.d/rpcbind start
```

服务端（LXC容器）：

```
/etc/init.d/nfs stop
```

```
/etc/init.d/rpcbind start
```

```
/etc/exports
```

```
cd /opt/software
```

```
wget http://nchc.dl.sourceforge.net/project/unfs3/unfs3/0.9.22/unfs3-0.9.22.tar.gz
```

```
tar xzf unfs3-0.9.22.tar.gz
```

```
cd unfs3-0.9.22
```

```
./configure
```

```
make && make install
```

启动：unfsd -d调试用 unfsd后台

客户端（可以是LXC容器）：

```
/etc/init.d/rpcbind start
```

```
mount -t nfs .....
```

5.25. 容器重启导致宿主终端报错

- 当容器运行时间较长（此次事件恰巧为容器运行3个月）时容器里使用reboot/shutdown等重启或关机命令导致宿主机任意终端（包括tty、pts）均出现以下报错

```
Message from syslogd@17173bj-test02 at Dec 4 15:31:38 ...
```

```
kernel:unregister_netdevice: waiting for lo to become free. Usage count = 3
```

- 这是rhel6.4的内核bug，暂时无法解决
- 避免此类问题解决方案：禁止容器内部通过reboot等命令进行重启或关机，即删除此类命令，已通过lxc-sysinit.conf解决：

```
start on startup
env container

pre-start script
    if [ "x${container}" != "xlxc" -a "x${container}" != "xlibvirt" ]; then
        stop;
    fi
    initctl start tty TTY=console
    rm -f /var/lock/subsys/*
    rm -f /var/run/*.pid
    rm -f /etc/mtab
    telinit 3
    if [ ! -e /etc/mtab ]; then
        echo '/dev/root / rootfs defaults 0 0' > /etc/mtab
        echo 'none /dev/shm tmpfs rw,nosuid,nodev 0 0' >> /etc/mtab
    else
        grep -q 'rootfs' /etc/mtab
        [ $? -eq 1 ] && echo '/dev/root / rootfs defaults 0 0' >> /etc/mtab
        grep -q 'tmpfs' /etc/mtab
        [ $? -eq 1 ] && echo 'none /dev/shm tmpfs rw,nosuid,nodev 0 0' >> /e
tc/mtab
        grep -q 'devpts' /etc/mtab
        [ $? -eq 0 ] && sed -i '/devpts/d' /etc/mtab
    fi
    rm -f /sbin/reboot /sbin/poweroff /sbin/halt /sbin/shutdown
    rm -f /etc/rc.d/rc0.d/* /etc/rc.d/rc6.d/*
    rm -f /usr/bin/halt /usr/bin/poweroff /usr/bin/reboot
    exit 0;
end script
```

5.26. lxc-start时候宿主内核crash

- 测试部反馈最近测试部服务器发生两起实体机服务器自动启停现象

实体机ip:192.168.2.3

现象:自动启停

时间:第一次发生在凌晨，当时无人为操作，第二次时间为:2014/05/05 17:30-18:00

人为操作:进行虚拟机创建、初始化操作，操作虚拟机过程中，实体机自动宕机，稍后机器自动恢复正常

实体机ip:192.168.2.4

现象:自动启停

时间:2014/05/06 11:30-12:00

人为操作:进行虚拟机启动操作,操作虚拟机过程中,实体机自动宕机,稍后机器自动恢复正常

- 根据沟通得知在lxc-start执行后没有任何输出,然后内核就crash了

192.168.3.1 crash是由宿主机的[netns]进程导致,192.168.3.2 crash是由宿主机的hald进程导致,在crash发生之前系统性能均无异常,通过ILO查看硬件状态和日志也没有发现任何异常情况。现初步怀疑是由红帽6内核bug所致,暂时无法解决,已将问题提交至红帽官方(bugzilla.redhat.com),等待答复。

- RHEL6.4内核转储及查看

RHEL6.4内核转储及查看

一. 安装包:

1. crash (通过yum安装)
2. kernel-debuginfo-2.6.32-358.el6.x86_64.rpm
3. kernel-debuginfo-common-x86_64-2.6.32-358.el6.x86_64.rpm

二. 开启kdump服务: /etc/init.d/kdump start

注:通过kickstart安装的RHEL6.4系统已默认开启kdump服务

确认kdump服务开启: /etc/init.d/kdump status,若输出Kdump is operational则表示kdump服务开启成功

三. 查看转储文件

当内核crash后,会在/var/crash/目录下生成转储文件,举例如下:

/var/crash/127.0.0.1-2014-05-07-15:24:33/

在该目录中,会生成vmcore和vmcore-dmesg.txt这2个文件,作用:

vmcore: 内核crash时进行内存转储

vmcore-dmesg.txt: 内核crash时将当时的dmesg信息保存至此

查看vmcore的命令: crash /usr/lib/debug/lib/modules/2.6.32-358.el6.x86_64/vmlinux . /vmcore

输入bt

图中的COMMAND: "crash_test.sh" 表示crash是由crash_test.sh程序触发的。

附:

1. 手动触发内核crash的方法 echo c > /proc/sysrq-trigger

2. crash> 交互式常用指令:

? 查看帮助,有很多命令,可以查看当时的进程、内存、模块等等

help bt 查看bt命令帮助

5.27. 容器内无法使用iptables(模块加载失败)

- 原因：默认情况下关闭容器的sys_module（LINUX能力），致使在容器内部无法加载和卸载模块。

这是出于安全考虑，防止一个容器内部做模块加载卸载操作影响其他容器

- 解决方法：在宿主上加载模块，容器即可使用

最简单的方法是在宿主上执行`iptables -L`，这样就会把相应模块都加载进来

宿主加载完成后在容器里也执行`iptables -L`查看是否正常输出，若报错*FATAL: Could not load*

*/lib/modules/2.6.32-358.el6.x86_64/modules.dep: No such file or directory*则执行：`ln -s`

/lib/modules/2.6.32-358.23.2.el6.x86_64 /lib/modules/2.6.32-358.el6.x86_64 是由于在做LXC容器模板时候是通过yum来创建模板的目录结构、软件等，而yum源实际是CentOS源

5.28. udev导致容器异常

- 容器不允许启动udev，但有时候在容器启动时候会启动，导致容器虽然启动成功，但无法登陆，查看容器进程发现启动了udev进程，解决方法：

```
rm -f /opt/lxc/xxx/rootfs/sbin/start_udev
sed -i 's:^(/sbin/start_udev\):#\1:' /opt/lxc/xxx/rootfs/etc/rc.d/rc.sysinit
```

已更新17173-lxc-utils脚本，在0.7.5-18版本中解决了该问题，容器container_init时候会进行上述处理

0

本条目发布于2013年11月14日 [http://17173ops.com/2013/11/14/linux-lxc-install-guide.shtml] 。属于系统运维分类，被贴了 Linux、LXC 标签。作者是173ops。

《LXC基础学习教程》上有1条评论

严勇平

2015年6月11日下午4:38

补充下

macvlan几种模式的详细解释:

<http://backreference.org/2014/03/20/some-notes-on-macvlanmacvtap/>
